MINI PROJECT

ACADEMIC PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

A Java Based Application with JDBC and MySQL



Java | JDBC | MySQL

Manage Students | Store Data | Build Smarter

SUBMITTED BY:

Name : Utkarsh Dixit

Roll No. : 2500270130204

Department : IT

Semester : 3rd

AJAY KUMAR GARG ENGINEERING COLLEGE

DEPARTMENT OF INFORMATION TECHNOLOGY

GHAZIABAD, UTTAR PRADESH

ACADEMIC YEAR

2026 - 2027

Declaration

I hereby declare that this project report entitled **“Student Management System”** is prepared as part of my academic mini project work. The application was developed using Java and MySQL with JDBC for database connectivity. The project demonstrates CRUD operations, input validation, persistent data storage, SQL queries and object-oriented programming concepts.

Name: Utkarsh Dixit

Roll No.: 2500270130204

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me during the development of this mini project. The project helped me understand how Java applications communicate with relational databases using JDBC and how CRUD operations can be implemented in a practical application. I also gained experience in project organization, SQL queries, exception handling, validation and testing.

Abstract

The Student Management System is a console-based Java application developed to manage student records efficiently. The system stores student information such as ID, name, age, course, email and phone number in a MySQL database. JDBC is used as the communication layer between Java and MySQL. The application provides facilities to add, view, update, delete and search student records. Input validation and delete confirmation are included to improve reliability.

Table of Contents

1	Introduction, Objectives & Problem Statement
2	Technologies & System Requirements
3	System Architecture & Data Flow
4	Database Design
5	Modules & Features
6	Implementation & CRUD Operations
7	Validation, Error Handling & Testing
8	Source Code Evidence
9	Program Outputs / Execution Evidence
10	Advantages, Limitations & Future Scope
11	Conclusion & References

1. Introduction

Student records are commonly maintained in educational institutions. A computerized management system makes it easier to store, retrieve and modify these records. This project provides a simple menu-driven console interface and uses Java for application logic, JDBC for connectivity and MySQL for persistent storage.

Objectives

• Create a simple student record management application.

• Store student information permanently in MySQL.

• Implement Create, Read, Update and Delete (CRUD) operations.

• Provide student search by unique ID.

• Validate important user inputs.

• Demonstrate Java classes, methods, JDBC and SQL.

Problem Statement

Manual student record management can become difficult as the number of records increases. Searching, updating and deleting records manually may take additional time and can lead to mistakes. The proposed system provides a structured database-backed solution through a menu-driven Java application.

2. Technologies Used

Technology	Purpose
Java 21	Application logic, OOP and console interface
JDBC	Communication between Java and MySQL
MySQL	Relational and persistent database storage
Maven	Project configuration and dependency management
IntelliJ IDEA	Development, running and debugging
SQL	Database creation and CRUD queries

System Requirements

• Hardware: basic computer with at least 4 GB RAM and sufficient storage.

• Software: Windows/Linux/macOS, JDK 21 or compatible Java version, MySQL Server, IntelliJ IDEA, Maven and MySQL Connector/J.

• Database: MySQL database named student_management with a table named students.

3. System Design and Architecture

The project follows a simple layered structure. Main handles the console menu and user input. Student represents the student entity. StudentDAO contains database operations such as insert, select, update, delete and search. DBConnection creates the JDBC connection to MySQL.

Layer / Class	Responsibility
Main.java	Menu, input handling, validation and workflow
Student.java	Student data model: ID, name, age, course, email, phone
StudentDAO.java	SQL/database operations using JDBC
DBConnection.java	Centralized MySQL JDBC connection
MySQL	Persistent storage of student records

Data Flow: User → Main → StudentDAO → DBConnection/JDBC → MySQL → ResultSet/status message → User.

System Data Flow

```
Java application
↓
JDBC Driver
↓
MySQL Server
↓
Result / Status Message
```

4. Database Design

Field	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique student ID
name	VARCHAR(100)	NOT NULL	Student name
age	INT	NOT NULL	Student age
course	VARCHAR(100)	NOT NULL	Course / program
email	VARCHAR(100)	NOT NULL	Student email
phone	VARCHAR(15)	Optional	Student phone number

SQL Table Creation

```
CREATE TABLE students (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
age INT NOT NULL,
course VARCHAR(100) NOT NULL,
email VARCHAR(100) NOT NULL,
phone VARCHAR(15)
);
```

The auto-increment ID acts as the unique identifier used for search, update and delete operations.

5. Modules and Features

Module	Function
Add Student	Accepts student details and inserts a new record into MySQL.
View Students	Displays all stored student records.
Update Student	Updates an existing student using ID.
Delete Student	Deletes a selected student after confirmation.
Search Student	Finds a student by unique student ID.
Exit	Closes the application safely.

Implementation classes: DBConnection.java creates the JDBC connection; Student.java defines the data model; StudentDAO.java implements database operations; Main.java provides the menu-driven interface, validation and calls DAO methods.

6. Implementation and CRUD Operations

CRUD Operations

Operation	SQL / Purpose
Create	INSERT query adds a new student.
Read	SELECT queries display all students or retrieve one by ID.
Update	UPDATE query changes selected student information.
Delete	DELETE query removes a selected student after confirmation.
Search	SELECT ... WHERE id=? finds one student.

Search Query Pattern

```
String sql = "SELECT * FROM students WHERE id=?";
PreparedStatement pst = con.prepareStatement(sql);
pst.setInt(1, id);
ResultSet rs = pst.executeQuery();
```

Main Menu Dispatch — submitted code structure

```
switch (choice) {
case 1: addStudent(); break;
case 2: viewStudents(); break;
case 3: updateStudent(); break;
case 4: deleteStudent(); break;
case 5: searchStudent(); break;
case 6: exit;
}
```

7. Validation, Error Handling and Testing

• Name and course fields are checked so they are not empty.

• Age is checked for a valid numeric range.

• Email uses basic @ and . validation.

• Phone numbers are checked for exactly 10 digits.

• Invalid integer input is handled so the application can request input again.

• SQL exceptions are caught and reported.

• Try-with-resources is used where applicable to close JDBC resources safely.

Test Case	Action	Expected Result	Status
TC-01	Add valid student	Student added successfully	PASSED
TC-02	View students	All stored records displayed	PASSED
TC-03	Search existing ID	Matching student displayed	PASSED
TC-04	Search non-existing ID	Student not found message	PASSED
TC-05	Update existing ID	Student updated successfully	PASSED
TC-06	Delete + Y	Student deleted	PASSED
TC-07	Delete + N	Delete operation cancelled	PASSED

8. Source Code Evidence

The submitted project source is organized into three supporting Java classes. The following pages preserve the supplied IntelliJ screenshots as source-code evidence. Password values are redacted in this report for security.

Student.java

Defines the student entity with ID, name, age, course, email and phone fields, constructors and accessors.

StudentDAO.java

Contains SQL/database operations and uses JDBC classes such as PreparedStatement and ResultSet.

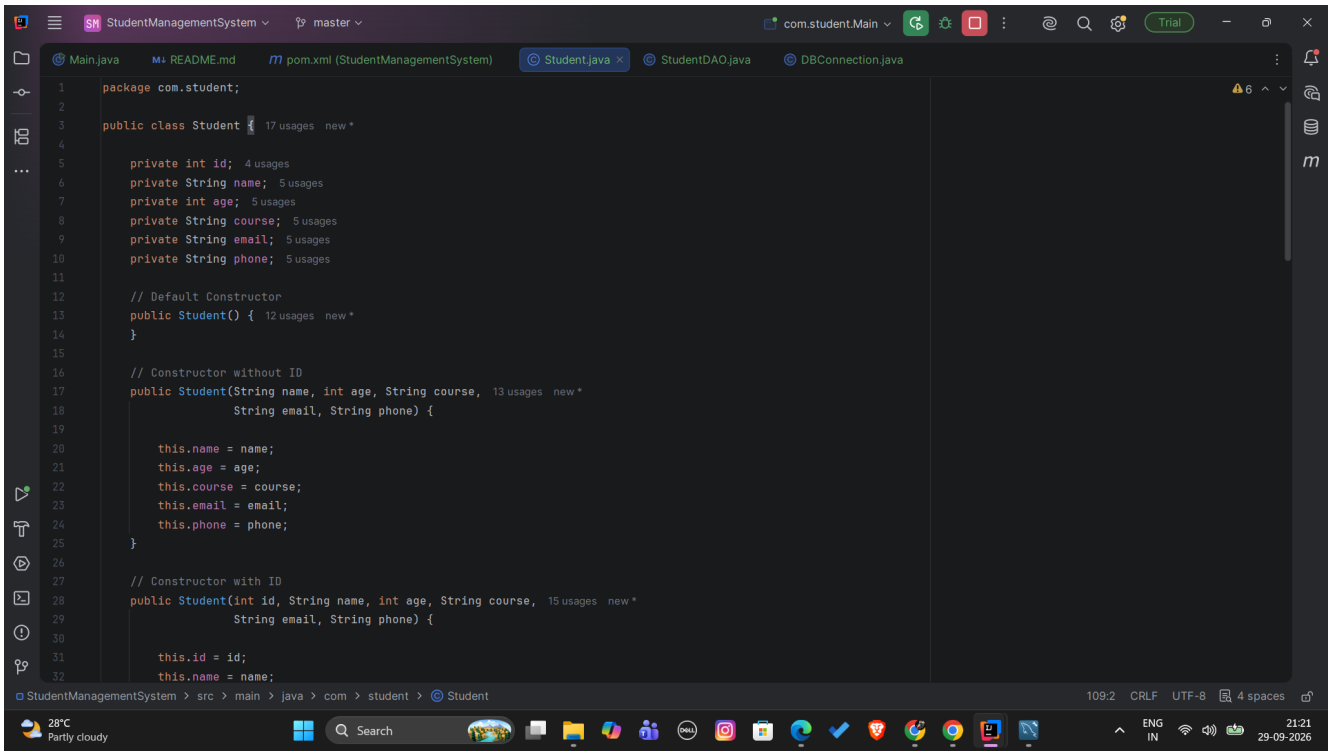
DBConnection.java

Creates the MySQL connection through DriverManager using the student_management database.

DBConnection.java — Password Redacted

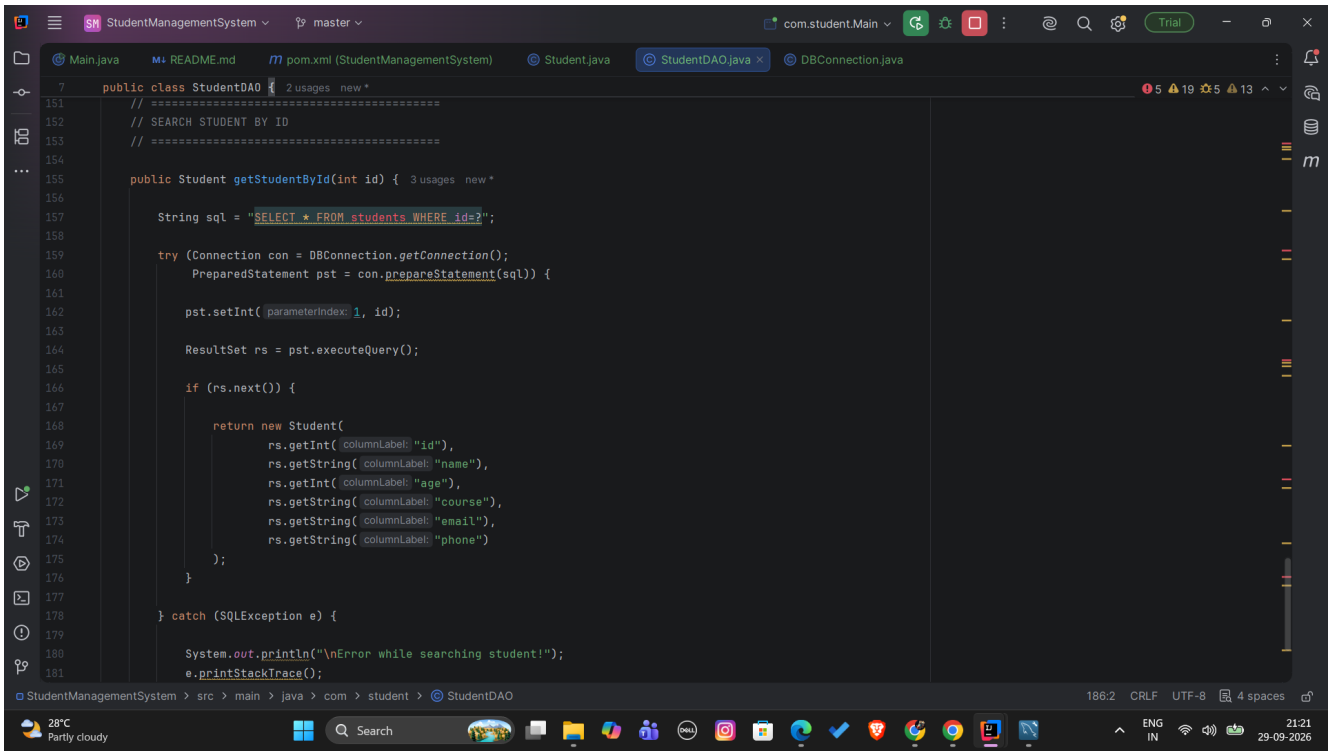
```
private static final String URL =
    "jdbc:mysql://localhost:3306/student_management";
private static final String USER = "root";
private static final String PASSWORD = "********";
Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
```

Student.java — Submitted Source Screenshot



Source-code evidence supplied from the IntelliJ IDEA project during development.

StudentDAO.java — Submitted Source Screenshot



```
public class StudentDAO {  2 usages  new *
    // ========================================
    // SEARCH STUDENT BY ID
    // ========================================

    public Student getStudentById(int id) {  3 usages  new *

        String sql = "SELECT * FROM students WHERE id=?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement pst = con.prepareStatement(sql)) {

            pst.setInt( parameterIndex: 1, id);

            ResultSet rs = pst.executeQuery();

            if (rs.next()) {

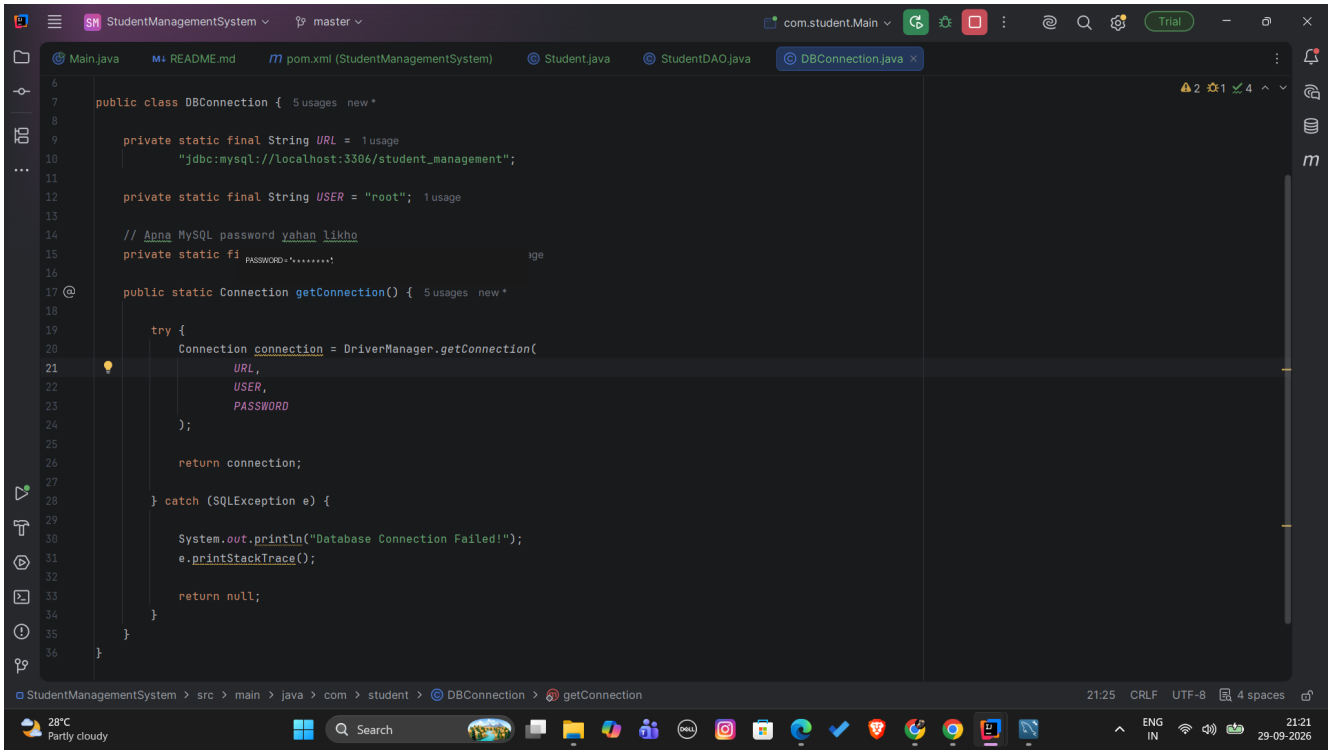
                return new Student(
                        rs.getInt( columnLabel: "id"),
                        rs.getString( columnLabel: "name"),
                        rs.getInt( columnLabel: "age"),
                        rs.getString( columnLabel: "course"),
                        rs.getString( columnLabel: "email"),
                        rs.getString( columnLabel: "phone")
                );
            }

        } catch (SQLException e) {

            System.out.println("\nError while searching student!");
            e.printStackTrace();
```

Source-code evidence supplied from the IntelliJ IDEA project during development.

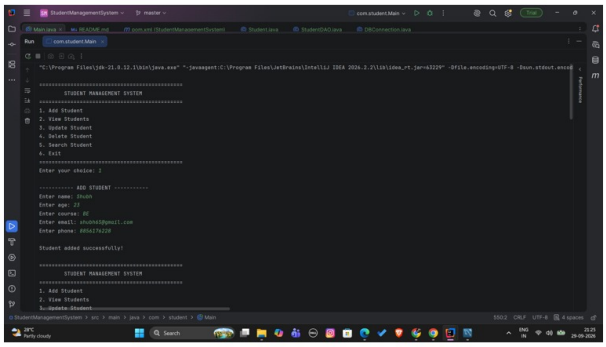
DBConnection.java — Submitted Source Screenshot (Password Redacted)

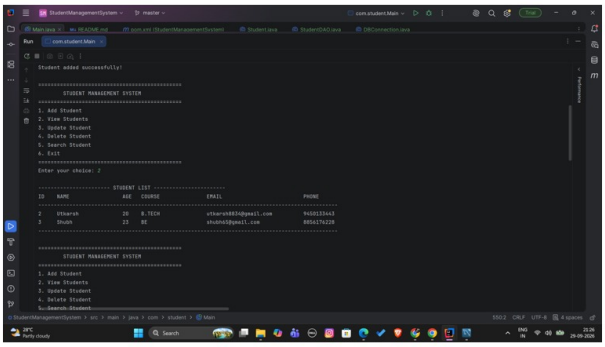


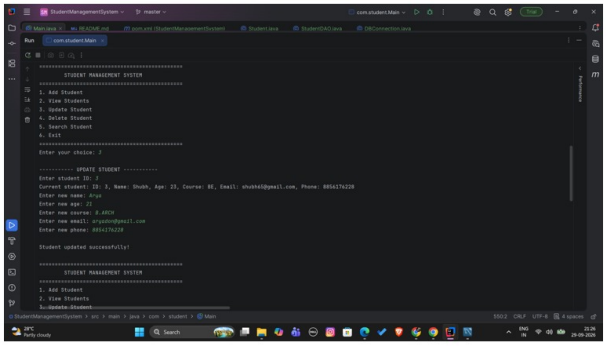
Source-code evidence supplied from the IntelliJ IDEA project during development.

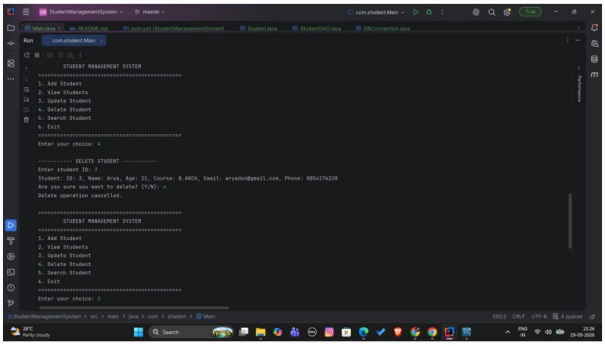
9. Program Outputs — Execution Evidence

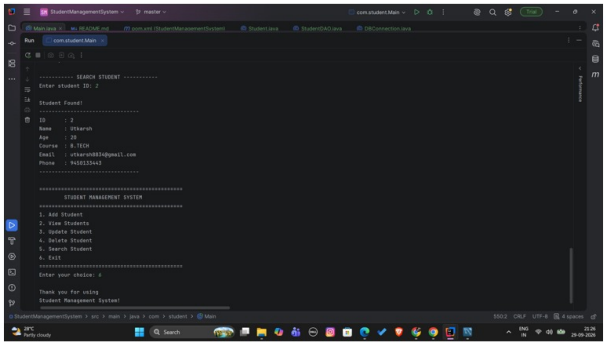
The following outputs were supplied from the running Java console application. They show the Add Student, View Students, Update Student, Delete Student and Search Student workflows.



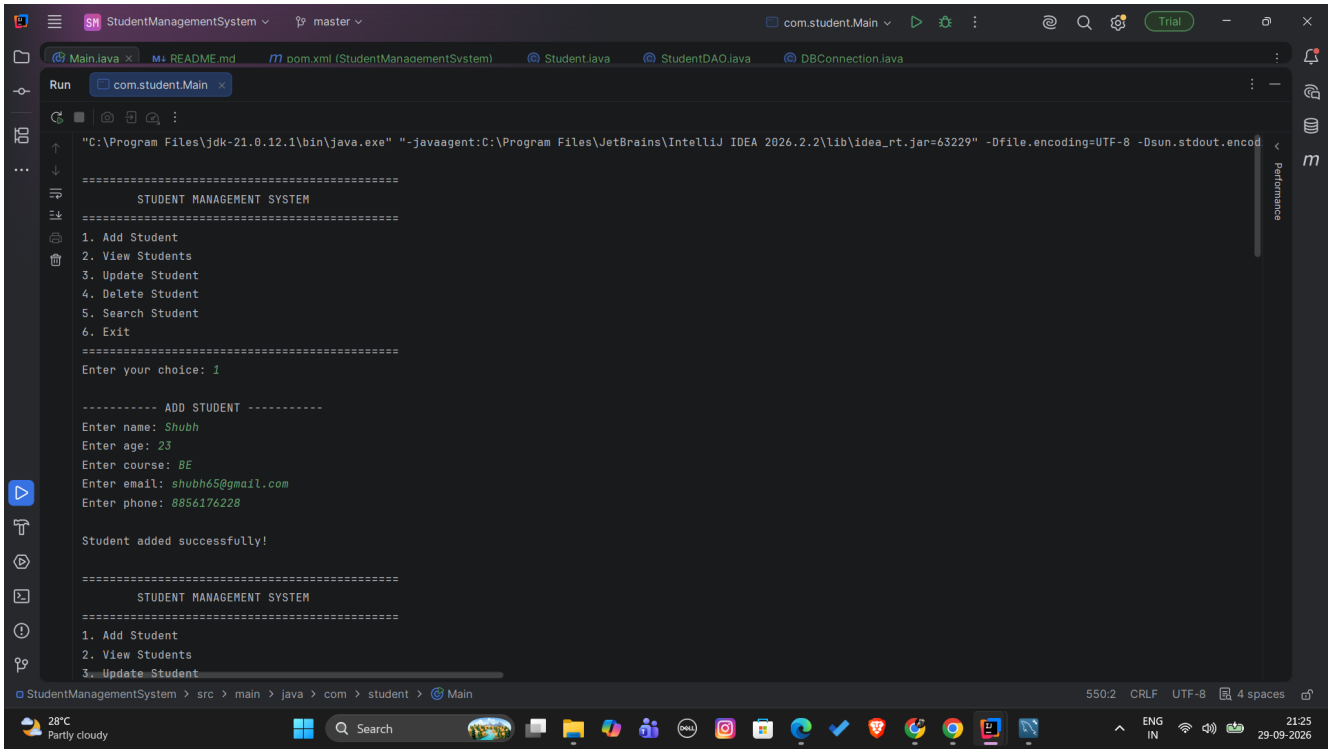




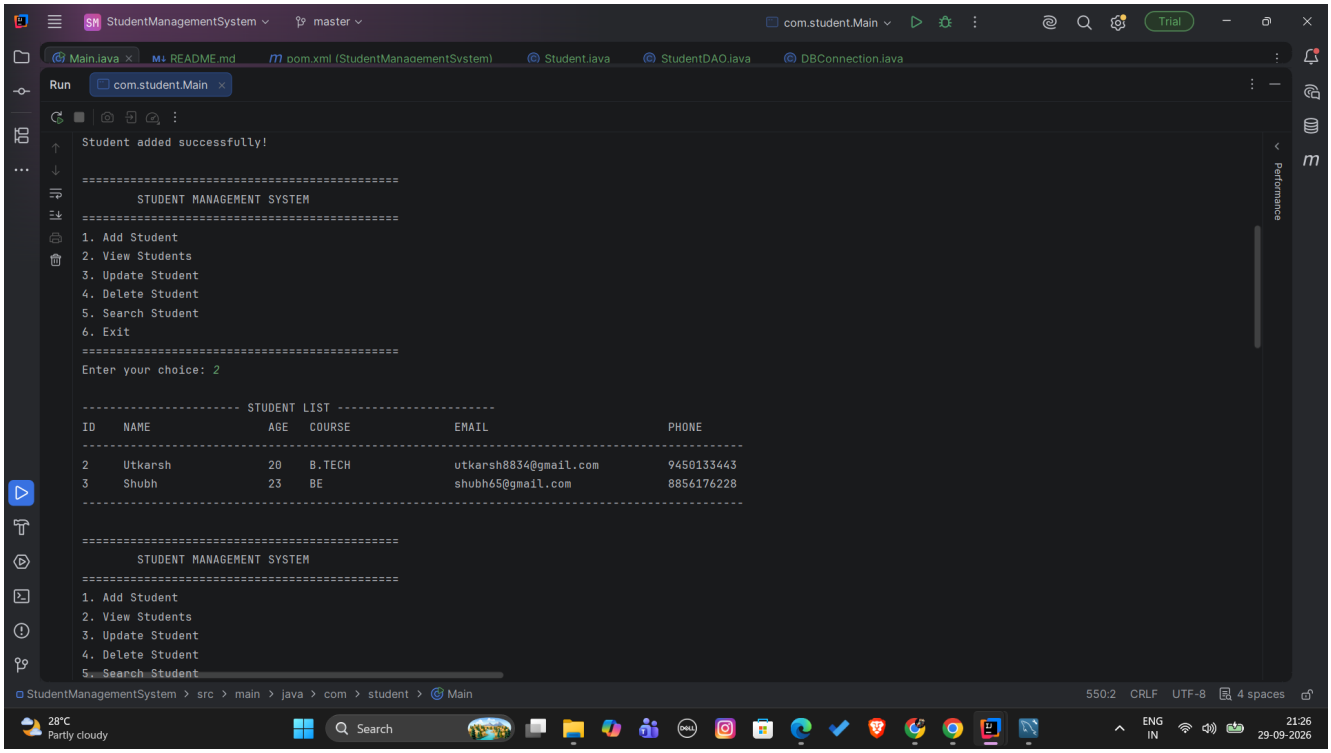




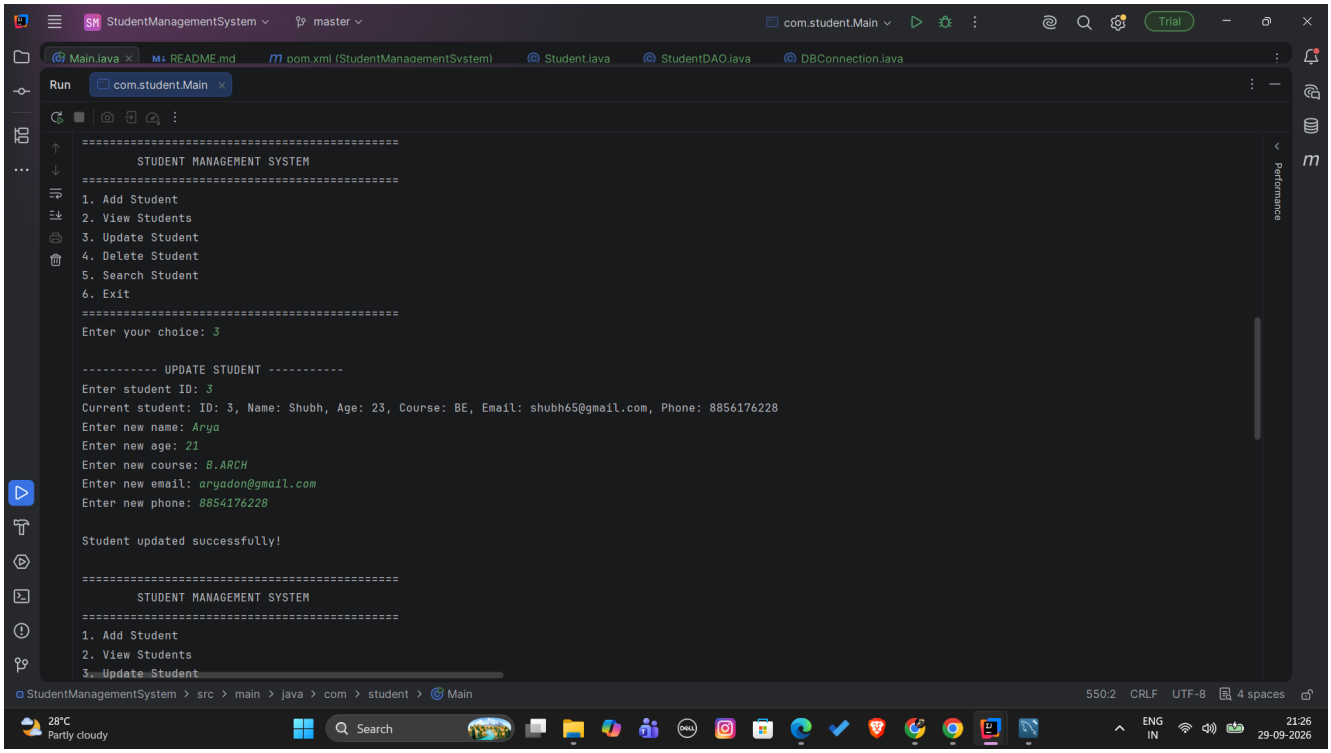
Add Student — Output



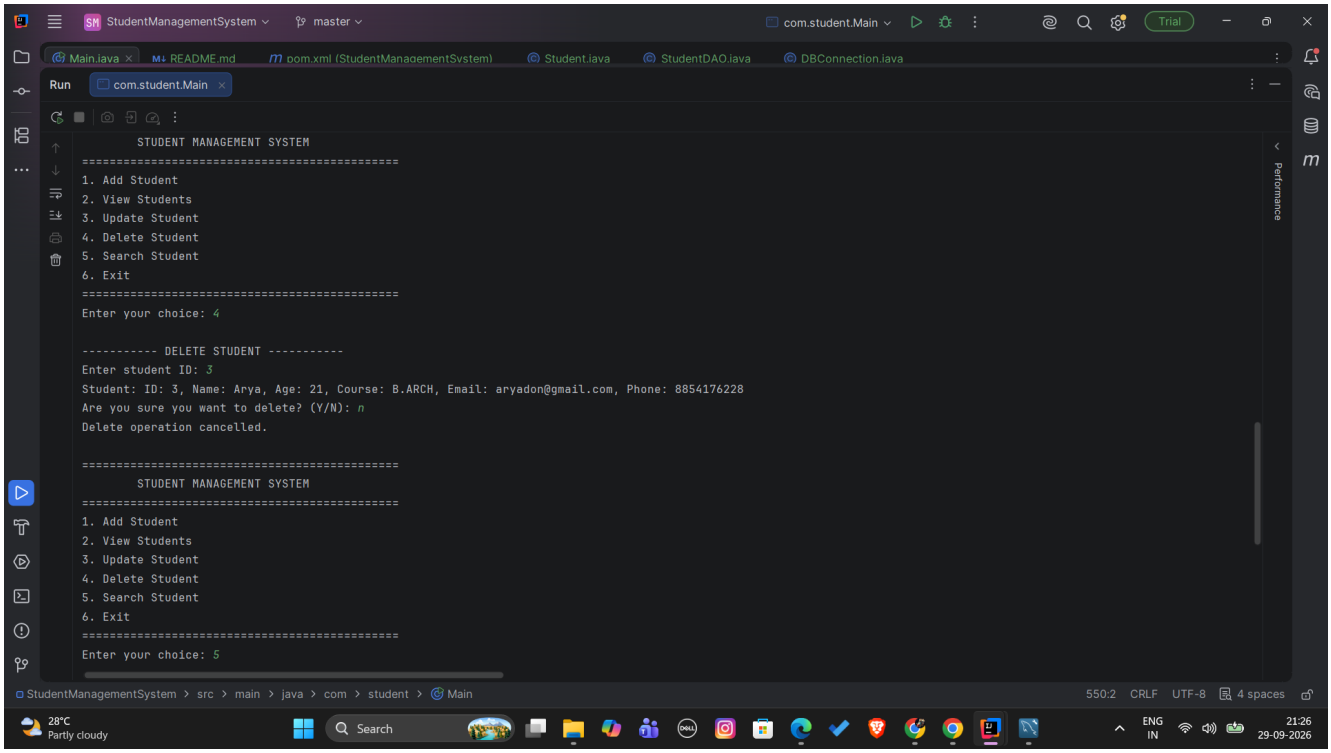
View Students — Output



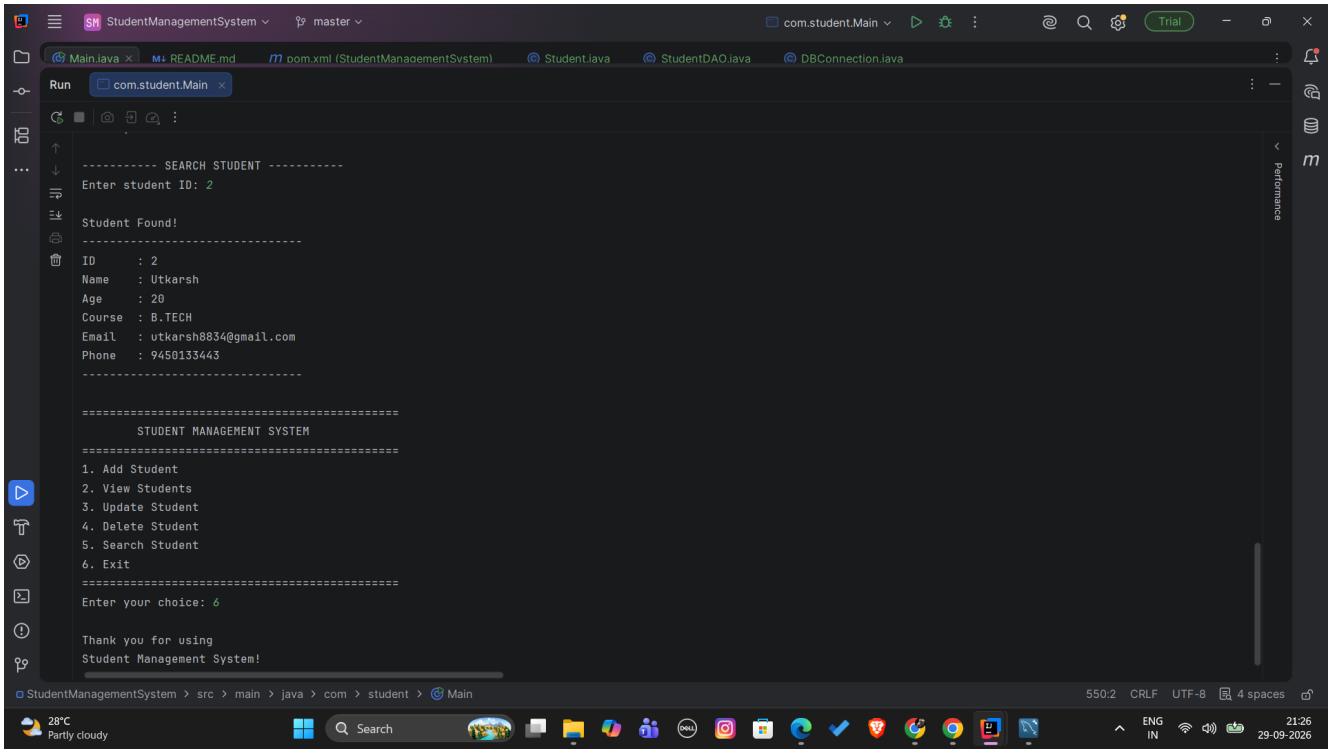
Update Student — Output



Delete Student — Output



Search Student / Exit — Output



10. Advantages, Limitations and Future Scope

Advantages

• Simple console interface.

• Persistent MySQL database storage.

• Organized class and DAO structure.

• CRUD functionality and student search.

• Input validation and exception handling.

• Reusable database-access methods.

Limitations

• Current version is console-based rather than GUI-based.

• Uses a local MySQL database.

• No authentication layer is included in the current version.

• Validation is basic and can be strengthened.

Future Scope

• JavaFX/Swing or web-based graphical interface.

• User login and role-based access.

• Advanced search, filtering and pagination.

• Report generation and export to PDF/Excel.

• Database backup and cloud deployment.

• Stronger validation and REST-based integration.

11. Conclusion

The Student Management System demonstrates how a Java application can interact with a MySQL database using JDBC. The project implements essential student record operations including adding, viewing, updating, deleting and searching records. It provides practical experience in Java programming, SQL, JDBC, object-oriented design, validation and exception handling.

References

• Oracle Java Documentation — Java and JDBC concepts.

• MySQL Documentation — SQL and database management concepts.

• MySQL Connector/J Documentation.

• Apache Maven Documentation.

• IntelliJ IDEA Documentation.

Project Snapshot: Student Management System • Java 21 • JDBC • MySQL • Maven • IntelliJ IDEA